Aula 3

Introdução a Machine Learning

Módulo 1: A Essência do ML

A transição da programação baseada em regras explícitas para o aprendizado derivado puramente de dados históricos.

Definição Fundamental

"Um programa de computador aprende com a experiência E..."

"...em relação a alguma classe de tarefas T e medida de desempenho P, se o seu desempenho em T, medido por P, melhora com a experiência E."

Tom Mitchell (1997)

Paradigmas do Aprendizado

Supervisionado

O modelo recebe as perguntas (Features) e as respostas certas (Target). O seu objetivo é mapear a relação matemática entre eles.

Não Supervisionado

Não há "respostas". O modelo recebe apenas os dados crus e procura autonomamente por padrões e grupos ocultos.

Por Reforço

O agente atua num ambiente, comete erros, ganha pontos ou penalizações, evoluindo através de tentativa e erro focado na recompensa.

Dicionário de Dados

Estrutura Tabular

- **Instância:** Uma linha do dataset. Representa um objeto único (um cliente, uma casa, um paciente).
- **Feature:** Variáveis de entrada (colunas). As características usadas para o modelo tomar a decisão.
- **Target:** Rótulo. A coluna que queremos prever no aprendizado supervisionado.

Processo

- **Treinamento (Fit):** O ajuste dos pesos matemáticos para minimizar o erro na base conhecida.
- **Generalização:** O sucesso do modelo em atuar bem quando exposto a dados do mundo real, nunca antes vistos.

Overfitting e Underfitting

Underfitting (Subajuste)

O modelo é "teimoso" e simples demais. Ele ignora as complexidades reais dos dados.

- **Exemplo:** Tentar ajustar uma reta em dados que fazem uma curva em "U".
- **Sintoma:** Métricas fracas no Treino E no Teste.

Overfitting (Sobreajuste)

O modelo possui muita "memória" e pouca "lógica". Ele decora todos os ruídos dos dados de treino.

- **Exemplo:** Uma linha extremamente ruidosa que liga todos os pontos perfeitamente.
- **Sintoma:** Acurácia 100% no Treino, mas destrói-se no Teste.

O Trade-off Viés-Variância

O objetivo supremo do Machine Learning é encontrar o equilíbrio entre flexibilidade e restrição estatística.

$$\text{Erro} = \text{Viés}^2 + \text{Variância} + \text{Erro}_{\text{Irredutível}}$$

Alto Viés: Causa Underfitting. Assunções incorretas sobre a base (Ex: Regressão Linear em dados polinomiais).

Alta Variância: Causa Overfitting. Modelos ultra-sensíveis que mudam drasticamente de acordo com a amostra utilizada.

Módulo 2: Domínio e Dados

Apresentando as bases clássicas da literatura que servem de benchmark para o ensino prático do scikit-learn.

Iris Dataset

Classificação Botânica

Um dos datasets mais famosos da estatística, introduzido por Ronald Fisher.

- **Features:** 4 colunas numéricas (Comprimento e largura de Sépalas e Pétalas).
- **Target:** 3 Espécies (Setosa, Versicolor, Virginica).
- **Características:** 150 amostras, perfeitamente balanceado (50 de cada), sem dados nulos.

California Housing

Regressão Imobiliária

Dataset extraído do censo da Califórnia de 1990. Perfeito para predição de valores contínuos.

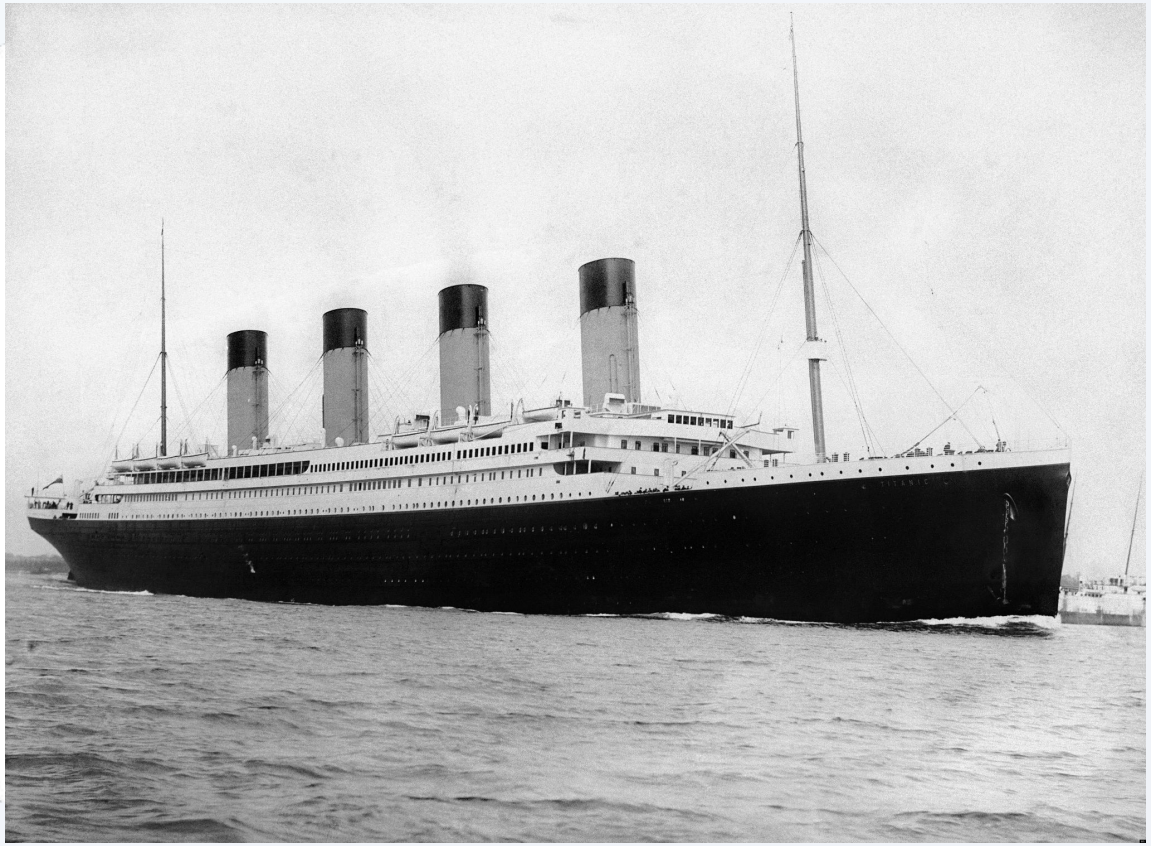
- **Features:** Rendimento mediano, idade da casa, quartos, população, latitude, longitude.
- **Target:** Preço mediano da casa do quarteirão.
- **Características:** 20.640 instâncias. Exige escalonamento severo devido a métricas de escalas variadas.

Titanic Dataset

Sobrevivência no Mar

O "Hello World" do Kaggle. O objetivo é prever a sobrevivência dos passageiros (Classificação Binária 0/1).

- **Features Numéricas:** Idade, Tarifa (Fare), Irmãos a bordo.
- **Features Categóricas:** Sexo, Classe (1ª, 2ª, 3ª), Porto de embarque.
- **Desafios:** Altíssima taxa de dados em falta (especialmente em 'Idade' e 'Cabine').



Módulo 3: Feature Engineering

A arte e a ciência de transformar dados brutos em representações matemáticas ótimas para algoritmos.

Seleção de Features (1/2)

A Maldição da Dimensionalidade

Adicionar muitas colunas aumenta o tempo de treino e, paradoxalmente, *piora* o modelo se as variáveis não adicionarem valor real (ruído).
Objetivo: Reduzir overfitting, melhorar a interpretabilidade e otimizar recursos computacionais descartando o lixo.

Métodos de Filtro (Filters)

Estatística pura. Independem do modelo preditivo que será usado.

- **ANOVA (f_classif):** Mede o quão bem uma feature contínua separa classes distintas.
- **Informação Mútua:** Mede a dependência geral, capturando inclusive fortes correlações não-lineares.

Seleção de Features (2/2)

Métodos Wrapper

O processo usa o próprio modelo final como "avaliador". Testa milhares de subconjuntos de variáveis.

RFE (Recursive Feature Elimination): Treina o modelo, analisa a "importância" das variáveis, rejeita a pior, e repete o treino até sobrar o "K" desejado. Extremamente exigente em processamento.

Métodos Embedded

A seleção é orgânica e acontece dentro da matemática do próprio algoritmo de treino.

- **Lasso (Regularização L1):** Otimizador que zera matematicamente os coeficientes de features inúteis.
- **Random Forest:** Calcula nativamente as features que mais reduzem a impureza nos nós.

Escalonamento: Normalização Min-Max

Reduz os dados de uma coluna para caberem rigidamente num intervalo específico, geralmente entre 0 e 1.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Atenção: Como a fórmula utiliza os limites físicos máximos e mínimos globais, a presença de um único **outlier gigante** esmagará 99% dos dados normais para muito próximo de zero.

Escalonamento: Padronização (Standard)

Converte a feature para o Z-Score. Transforma a coluna para ter Média = 0 e Desvio Padrão = 1.

$$X_{std} = \frac{X - \mu}{\sigma}$$

Vantagem: É o "padrão ouro" para a grande maioria dos algoritmos (SVM, Logística, Redes Neurais). Diferente do Min-Max, não estabelece um limite rígido (podem existir valores negativos e > 1).

Robust Scaler e Outliers

O Problema da Média

Tanto o *StandardScaler* (Média) quanto o *MinMaxScaler* (Extremos) usam estatísticas que são distorcidas brutalmente por valores anómalos.
Se tivermos salários de $2k, $3k e, do nada, $100M (outlier), a média fica completamente distorcida.

A Solução Robusta

O RobustScaler ignora a média e o desvio padrão. Ele usa a **Mediana** (valor central) e o **IQR** (Intervalo Interquartil - do 25º ao 75º percentil).
Ao guiar-se pelo centro de massa, ignora os extremos absurdos, salvando as distâncias reais da distribuição.

Automação: Pipelines no sklearn

Prevenção de Data Leakage O Pipeline assegura que os métodos de imputação e escalonamento aprendem (fit) estritamente sobre a fatia de Treino durante a Validação Cruzada, nunca espreitando os dados de Teste/Validação.

Simplicidade Sintática Em vez de chamar fit_transform() para NaNs, depois para Escalonamento, depois para o Modelo... O Pipeline empacota tudo num objeto só. Basta chamar pipe.fit(X_train, y_train).

ColumnTransformers

Caminhos Independentes

Datasets não são uniformes. O Titanic possui Idade (Numérico) e Sexo/Porto (Categórico).

Não pode passar o "StandardScaler" na coluna 'Sexo' e não pode aplicar "OneHotEncoding" na coluna 'Idade'.

Como Funciona

O ColumnTransformer divide o dataset pelo nome da coluna, aplica sub-pipelines especializados, e no final realiza uma concatenação automática, gerando uma matriz de Treino única, processada e pronta.

Data Augmentation Tabular

O Algoritmo SMOTE

Synthetic Minority Over-sampling Technique. Resolve o desbalanceamento brutal de classes.

Ele rastreia exemplos da classe rara e cria pontos artificiais interpolados matematicamente na distância entre o exemplo original e os seus K vizinhos.

Ruído Gaussiano

Uma técnica de regularização criativa.

Adicionamos micro-perturbações aleatórias nas features (ex: somar 0.05 a uma idade). Obriga o modelo a não decorar os valores exatos da tabela, promovendo generalização rígida.

Módulo 4: Validação de Modelos

A prova de fogo. Técnicas indispensáveis para blindar a análise contra métricas ilusórias e garantir robustez industrial.

Treino, Validação e Teste

O Split Simples

Separamos os dados usando train_test_split.

- **Treino:** Usado exaustivamente para calcular gradientes e pesos.
- **Teste (Hold-out):** A prova final. Base trancada a sete chaves, usada apenas 1x no final do projeto para atestar o desempenho real sem viés.

O Papel da Validação

E se quisermos testar qual hiperparâmetro (ex: Profundidade da Árvore) é melhor?

Criamos um terceiro conjunto: **Validação**. Ajustamos parâmetros avaliando nele. Isso impede que o ajuste de parâmetros se vicie e vaze para o Teste Final.

K-Fold Cross-Validation

Robustez contra Amostras Sortudas Um único corte de treino/teste pode, por sorte, colocar casos fáceis no teste. O Cross-Validation anula a sorte analisando o dataset inteiro de várias perspetivas.

A Mecânica K-Fold A base é fatiada em K blocos (ex: 5). O modelo treina em 4 blocos e testa no que sobrou. O processo roda 5 vezes, testando com um bloco diferente em cada vez.

O Veredito A performance real do modelo é a **Média** das 5 rodadas, e a instabilidade do modelo é lida pelo **Desvio Padrão** das 5 rodadas.

A Importância do Stratified K-Fold

O Problema do Aleatório

Em datasets desbalanceados (Ex: 90% Cães, 10% Gatos), um K-Fold puramente aleatório pode gerar um "pedaço" com 100% Cães e 0% Gatos. O modelo entra em colapso.

A Trava da Estratificação

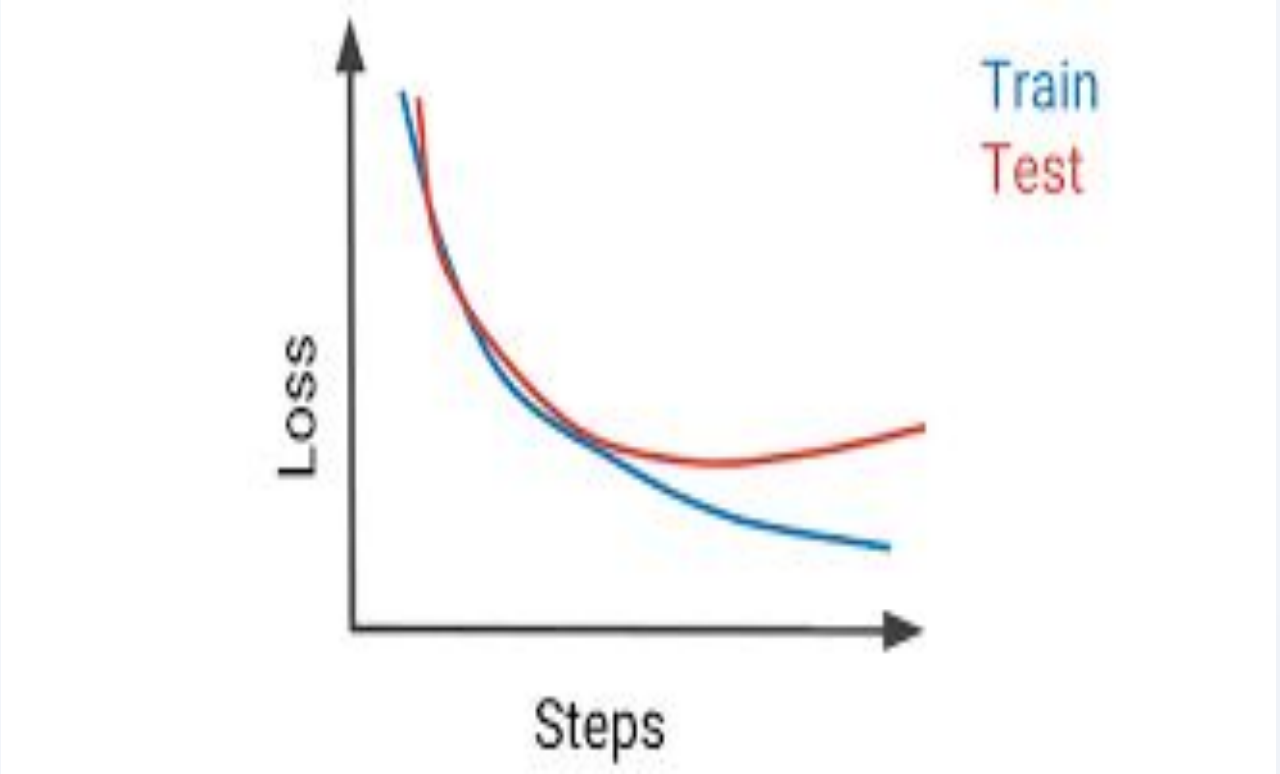
O StratifiedKFold audita os blocos no momento da divisão e impõe que cada "pedaço" do K-Fold tenha exatamente a mesma distribuição global (90/10). Vital para Classificações!

Curvas de Aprendizado (Learning Curves)

Diagnóstico Visual Avançado

Traça a performance no Treino e na Validação variando o volume de amostras que o modelo pode observar.

- **Alta Aderência:** Curvas fixam-se num patamar alto = Ótimo ajuste.
- **Gap Gigante:** Treino excelente, Validação péssima (gap largo) = Sofre de Overfitting brutal. Precisa de regularização.
- **Baixa Aderência:** Curvas colam-se muito cedo, mas em pontuações miseráveis = Underfitting.



Módulo 5: Algoritmos Preditivos

A matemática aplicada. Exploração teórica dos pesos, distâncias e florestas probabilísticas do Machine Learning clássico.

Regressão Linear Múltipla

O algoritmo base. Assume uma combinação aditiva rígida das features para aproximar o target minimizando resíduos (OLS).

$$\hat{y} = w_0 + w_1 x_1 + \dots + w_p x_p$$

Vulnerabilidades: Pode sofrer de multicolinearidade severa, onde variáveis redundantes destroem a estabilidade da equação de pesos.

Regularização: Penalidade Lasso (L1)

A Regularização altera a Função Custo, punindo o modelo se ele se tornar matematicamente complexo demais. O Lasso pune pelo valor absoluto.

$$\text{Custo} = \text{MSE} + \alpha \sum_{j=1}^{p} |w_j|$$

Seleção Automática: A geometria do Lasso (L1) força matematicamente atributos fracos ou redundantes a receberem peso *exatamente ZERO*. O Ridge (L2) apenas os reduz.

Algoritmos de Distância: KNN

Lazy Learning (KNN)

O K-Nearest Neighbors não possui "função de treino". Ele empilha a base na memória RAM.

Na inferência, ele mede a distância vetorial do novo elemento contra todos na base. A classe é o voto dos K vizinhos mais colados a ele.

Hiperparâmetros Chave

- **n_neighbors (K):** Valores de K pequenos tendem ao Overfitting. Valores massivos puxam a previsão sempre para a classe dominante (Viés).
- **Escalonamento:** Obrigatório usar StandardScaler. Features grandes distorcem o vetor de distância.

Support Vector Machines (SVM)

O Hiperplano Máximo

A meta do SVM é implacável: encontrar uma linha (ou plano em multi-dimensões) que separe categorias impondo a *Maior Margem* possível (uma "rua" larga e vazia entre as classes).

Parâmetro C (Regularização)

C alto: Punição dura! Tenta encaixar todos os pontos de forma perfeita, arriscando overfitting.

C baixo: Permite tolerância. A margem fica mais suave e aceita erros isolados no treino em troca de paz na generalização.

O Truque do Kernel (SVM)

O Problema da Linearidade Um hiperplano é uma linha reta. Se a distribuição de duas classes no papel formar círculos concêntricos (um anel dentro de outro), nenhuma linha reta poderá separá-los sem falhar em 50%.

A Mágica RBF e Polinomial A Função Kernel decola os dados matematicamente da folha 2D em direção a 3D, esticando as instâncias geometricamente até que um corte "plano" perfeitamente linear consiga separar o anel interno do externo!

Regressão Logística: O Log-Loss

Apesar do nome, é o rei da *Classificação*! Passa a fórmula linear por uma função Sigmoide (S), espremendo qualquer resultado num intervalo contínuo de [0, 1] (Probabilidade).

Função Sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Seja Limiar (Threshold) = 0.5. Probabilidades superiores predizem a Classe 1, inferiores a Classe 0. Usa a função custo Entropia Cruzada para corrigir probabilidades confusas.

Teorema de Naive Bayes

A Premissa "Ingénua"

Baseado na condicional de Bayes. O algoritmo pressupõe categoricamente que todas as variáveis são mutuamente independentes dada a classe avaliada.
Na prática, sabemos que features se afetam (ex: quartos de casa afeta a área quadrada), mas a matemática ignora isso.

Vantagens Táticas

- **Alta Escalabilidade:** Sendo cálculos probabilísticos simples, processa incrivelmente rápido sem iterar dezenas de epochs.
- **Domínio:** Rainha dos algoritmos da NLP antiga (Filtros de Spam baseados na contagem de dicionários: MultinomialNB).

Árvores de Decisão (Critério Gini)

Cortam os dados em regiões via regras IF/ELSE. A procura pela quebra foca em purificar os grupos, minimizando o índice Gini (Impureza).

$$\text{Gini} = 1 - \sum_{k=1}^{K} p_k^2$$

Overfitting Nativo: Uma árvore que cresce sem amarras (max_depth=None) ramificará até criar uma folha exclusiva para CADA LINHA do treino. Exige hiperparâmetros de "poda" antecipada.

Random Forest (A Força do Bagging)

O Conceito de Ensemble Uma árvore isolada é fraca e viciada (alta variância). Uma "Floresta" gera 100, 500, 1000 árvores e usa a democracia (voto maioritário) na predição.

Bootstrap Aggregation (Bagging) Se todas as árvores virem os mesmos dados, pensarão da mesma forma. O Random Forest resolve isso sorteando linhas aleatórias com repetição (Bootstrap) e um conjunto parcial de colunas para CADA árvore treinada!

Cancelamento de Erro Como as árvores são não-correlacionadas, os overfits isolados de uma árvore são diluídos pelas respostas da maioria. O resultado é o rei da estabilidade tabular.

Gradient Boosting e XGBoost

Arquitetura Serial

Ao invés da democracia paralela do Random Forest, o Boosting é serial, punitivo e focado no erro.

A Árvore 1 treina. A Árvore 2 não olha para o Target, olha para os *resíduos* (erros) que a Árvore 1 cometeu e tenta prever o erro, somando-se a ela. E assim por diante.

O Padrão da Indústria

XGBoost / LightGBM: Motores de Boosting injetados com regularização L1/L2 intensa na construção dos nós e paralelização no processador. Dominam rankings de performance e competições Kaggle do mundo tabular moderno.

Módulo 6: Avaliação

Traduzindo equações obscuras e abstrações em relatórios, pontuações acionáveis e garantias de negócio.

Métricas Base de Classificação

Precisão (Precision)

De todas as vezes que assinalei "É Fraude!", em quantas estava certo?

VP / (VP + FP)

Penaliza o Alarme Falso gravemente.

Recall (Sensibilidade)

De todas as reais "Fraudes" que aconteceram, quantas o meu modelo conseguiu achar e capturar?

VP / (VP + FN)

Foco na captura médica/criminal.

F1-Score

Uma média harmônica entre a Precisão e o Recall. Útil para sumarizar o score em datasets imensamente desbalanceados onde a 'Acurácia' é cega e enganadora.

A Famosa Curva ROC (AUC)

Trade-off no Threshold

Gráfico que correlaciona a Taxa de Verdadeiros Positivos (Eixo Y) com a Taxa de Falsos Positivos (Eixo X) conforme alteramos a tolerância percentual [0,1] para prever a classe positiva.
AUC = Área Sob a Curva: Uma nota matemática de capacidade de separação. 0.50 significa que o modelo prevê ao acaso. 1.0 significa clareza de fronteira absoluta.

Curvas Precisão-Recall

A Curva ROC pode inflar otimismo se a classe for ultra-minoritária. Nessas horas, substituímos a ROC plotando diretamente a curva PR, que avalia exaustivamente apenas a minoria de interesse, destilando o seu verdadeiro desempenho.

Métricas Otimizadas para Regressão

Lidando com variáveis contínuas (Preços, Temperatura, Câmbio).

Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Diferenças: O **MAE** calcula o erro linear simples e compreensível, resistente a outliers. O **MSE** eleva os erros ao quadrado, punindo severamente anomalias gigantes de previsão. O **R^2 Score** normaliza o aprendizado percentual (entre 0 e 1) informando quanta variância da resposta foi explicada matematicamente pelo algoritmo final.

Exercicio final está no notebook da aula